

Curso técnico em Desenvolvimento de Sistemas – 3º ano Ensino Médio
Disciplina Desenvolvimento de Sistemas – Professor Mizael Carlos

Atividade – CRUD Completo de Funcionário com Laravel 12

Contextualização

Você continua atuando como estagiário(a) do setor de Automação Industrial da empresa Metalúrgica Boa Vista Ltda. O Sistema de Cadastro de Equipamentos Industriais, construído com Laravel 12, já está parcialmente pronto: os módulos de Setor e de Equipamento foram concluídos, com Model, Migration/SQL, rotas, Controller e views Blade totalmente funcionais.

Agora o setor de Recursos Humanos solicitou um novo módulo: o cadastro de Funcionários, para registrar quem trabalha em cada setor da fábrica. Você ficou responsável por construir esse CRUD do zero, seguindo exatamente o mesmo padrão já utilizado nos módulos de Setor e Equipamento. A tabela funcionarios deve conter os campos id, nome, matricula, cargo.

Ao longo desta atividade, você implementará o CRUD de Funcionário em quatro etapas, nesta ordem: Cadastro (Create), Listagem (Read), Edição (Update) e Exclusão (Delete). Cada etapa só deve começar depois que a anterior estiver concluída e comprovada. Para cada questão, comprove sua implementação com uma evidência (print de tela) mostrando o comando executado no terminal (quando houver) e o conteúdo do arquivo de código correspondente — Model, Controller, rota ou template Blade, além do print da tela do navegador funcionando.

Lembrete — estrutura da tabela funcionarios

```
CREATE TABLE funcionarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(150) NOT NULL,  
  matricula VARCHAR(20) NOT NULL UNIQUE,  
  cargo VARCHAR(100) NOT NULL  
);
```

Instruções gerais

1. Utilize o mesmo projeto Laravel já usado para os CRUDs de Setor e Equipamento — não crie um projeto novo nem apague o que já existe.
2. Siga as etapas na ordem: Cadastro → Listagem → Edição → Exclusão. Não inicie uma etapa sem concluir e comprovar a anterior.
3. Em cada questão, implemente exatamente o arquivo pedido (Model, rota, Controller ou view) antes de tirar o print de evidência.
4. Sempre que a questão pedir um comando Artisan, execute-o no terminal do seu projeto e comprove com print mostrando o comando digitado e o retorno.
5. Cada evidência deve mostrar claramente o comando executado no terminal (quando aplicável) e/ou o conteúdo do arquivo de código correspondente.
6. Cole a imagem diretamente no espaço tracejado indicado como "EVIDÊNCIA" logo abaixo de cada questão.
7. Não é permitido apagar ou sobrescrever os CRUDs de Setor e Equipamento já existentes — o CRUD de Funcionário deve conviver com eles no mesmo projeto.
8. Use exatamente os nomes de tabela, campos, rotas e métodos indicados no enunciado, para manter o padrão do projeto.

Lista de comandos que serão praticados nesta atividade:

```
SQL CREATE TABLE · php artisan make:model · php artisan make:controller --resource ·  
Route::resource · php artisan route:list · Eloquent (all, create, update, delete) ·  
@foreach · @csrf · @method('PUT') · @method('DELETE')
```

Etapa 1 — Cadastro (Create)

Você vai construir, passo a passo, o caminho completo que salva um novo funcionário: tabela → Model → rota → Controller → formulário.

Comando(s)/Arquivo(s) envolvido(s): **SQL — CREATE TABLE**

Cenário — Preparando a tabela de funcionários

O RH aprovou o novo módulo de Funcionários. Antes de qualquer código PHP, é preciso preparar o banco de dados, seguindo o mesmo padrão usado nas tabelas setores e equipamentos.

Tarefa: Crie a tabela funcionarios via SQL, com os campos id, nome, matricula (único), cargo e setor_id como chave estrangeira para setores(id). Execute o comando no seu cliente SQL (MySQL Workbench, DBeaver ou phpMyAdmin).

 **EVIDÊNCIA** (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **php artisan make:model Funcionario**

Cenário — Criando o Model Funcionario

Com a tabela criada, é hora de dar ao Laravel uma classe PHP capaz de conversar com ela — exatamente como foi feito para Setor e Equipamento.

Tarefa: Execute `php artisan make:model Funcionario` e defina o array `$fillable` com os campos nome, matricula, cargo e setor_id.

 **EVIDÊNCIA** (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **Route::resource**

Cenário — Definindo as rotas do módulo

Antes de criar o Controller, é preciso reservar as URLs que o módulo de Funcionários vai usar, seguindo o padrão REST já adotado no projeto.

Tarefa: No `routes/web.php`, adicione `Route::resource('funcionarios', FuncionarioController::class)`. Em seguida, rode `php artisan route:list` e confirme que as 7 rotas do recurso foram geradas.

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): `php artisan make:controller --resource`

Cenário — Criando o Controller e o método store()

Com o Model e as rotas prontos, chegou a hora de implementar a lógica que efetivamente salva um novo funcionário no banco.

Tarefa: Execute `php artisan make:controller FuncionarioController --resource`. Implemente o método `store()`, validando nome, matrícula (obrigatório e único) e cargo como obrigatórios, e salvando o registro com `Funcionario::create()`.

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): `resources/views/funcionarios/create.blade.php`

Cenário — Construindo o formulário de cadastro

O back-end já sabe salvar um funcionário, mas falta a tela para o usuário preencher os dados.

Tarefa: Crie a view `create.blade.php` com um formulário Bootstrap contendo os campos nome, matrícula e cargo, além de um `<select>` com os setores cadastrados. Use `@csrf` e `action="{{ route('funcionarios.store') }}"`.

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **Teste de ponta a ponta**

Cenário — Testando o cadastro

Chegou a hora da verdade: cadastrar o primeiro funcionário pelo formulário e confirmar que ele realmente foi parar no banco de dados.

Tarefa: Preencha o formulário com os dados de um funcionário real (ex.: um técnico do setor de Manutenção) e envie. Confirme o registro salvo (via Dbeaver ou a listagem, se já tiver acesso a ela).

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Etapa 2 — Listagem (Read)

Com funcionários já cadastrados, é hora de exibí-los em uma tabela, seguindo o padrão de Setor e Equipamento.

Comando(s)/Arquivo(s) envolvido(s): **Controller – método index()**

Cenário — Implementando a listagem

Agora que já existem funcionários cadastrados, o RH quer poder vê-los todos em uma única tela.

Tarefa: Implemente o método index() no FuncionarioController, buscando os registros com Funcionario::all() e retornando a view funcionarios.index com os dados.

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **resources/views/funcionarios/index.blade.php**

Cenário — Exibindo os funcionários em tabela

Seguindo o mesmo padrão adotado nos módulos de Setor e Equipamento, a listagem deve ser uma tabela com uma coluna de Ações.

Tarefa: Crie a view index.blade.php como uma <table> com as colunas Nome, Matrícula, Cargo, Setor e Ações, usando @foreach para percorrer \$funcionarios. Na coluna Ações, deixe reservado o espaço para os botões Editar e Excluir (serão implementados nas próximas etapas).

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **Rota funcionarios.index**

Cenário — Conferindo a listagem no navegador

Com o Controller e a view prontos, é hora de ver o resultado funcionando de ponta a ponta.

Tarefa: Acesse a rota de listagem no navegador (ex.: /funcionarios) e confirme que o funcionário cadastrado na Etapa 1 aparece corretamente na tabela.

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Etapa 3 — Edição (Update)

Agora o sistema precisa permitir corrigir os dados de um funcionário já cadastrado.

Comando(s)/Arquivo(s) envolvido(s): **Controller – métodos edit() e update()**

Cenário — Permitindo a correção de dados

O RH percebeu um erro de digitação no cargo de um funcionário e pediu para você permitir a edição dos dados.

Tarefa: Implemente edit(Funcionario \$funcionario), que retorna a view de edição já com o registro, e update(Request \$request, Funcionario \$funcionario), que valida e atualiza os dados com \$funcionario->update().

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **resources/views/funcionarios/edit.blade.php**

Cenário — Construindo o formulário de edição

O formulário de edição deve ser igual ao de cadastro, mas já vindo preenchido com os dados atuais do funcionário.

Tarefa: Crie a view edit.blade.php reaproveitando os campos do create.blade.php, mas com os values preenchidos (ex.: value="{{ \$funcionario->nome }}"), method="POST" + @method('PUT'), e action="{{ route('funcionarios.update', \$funcionario) }}").

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **Coluna de Ações – botão Editar**

Cenário — Ligando a listagem à edição

Falta o último passo para fechar o ciclo de edição: o botão que leva do funcionário na listagem até o formulário de edição dele.

Tarefa: Na coluna Ações da view index.blade.php, adicione o link , estilizado como botão Bootstrap (ex.: btn btn-primary btn-sm), com o texto "Editar".

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Etapa 4 — Exclusão (Delete)

Por fim, o sistema precisa permitir remover um funcionário cadastrado.

Comando(s)/Arquivo(s) envolvido(s): **Controller – método destroy()**

Cenário — Permitindo a remoção de um cadastro

Um funcionário foi desligado da empresa e o RH pediu para remover o cadastro dele do sistema.

Tarefa: Implemente o método `destroy(Funcionario $funcionario)` no Controller, chamando `$funcionario->delete()` e redirecionando de volta para a listagem.

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **Coluna de Ações – botão Excluir**

Cenário — Adicionando o botão de exclusão

Assim como Editar, o botão Excluir precisa aparecer na coluna de Ações de cada linha da tabela.

Tarefa: Na coluna Ações, adicione um `<form>` com `method="POST"`, `@csrf`, `@method('DELETE')` e `action="{{ route('funcionarios.destroy', $funcionario) }}"`, contendo um botão "Excluir" (ex.: `btn btn-danger btn-sm`).

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)

Comando(s)/Arquivo(s) envolvido(s): **CRUD completo**

Cenário — Testando a exclusão e conferindo o CRUD completo

Com Cadastro, Listagem, Edição e Exclusão implementados, o módulo de Funcionários chega ao mesmo padrão dos módulos de Setor e Equipamento.

Tarefa: Exclua o funcionário de teste pelo botão Excluir e confirme que ele desaparece da listagem. Em seguida, rode `php artisan route:list` novamente e confirme que as 7 rotas de funcionarios continuam ativas.

 EVIDÊNCIA (cole aqui o print de tela do comando/código executado)